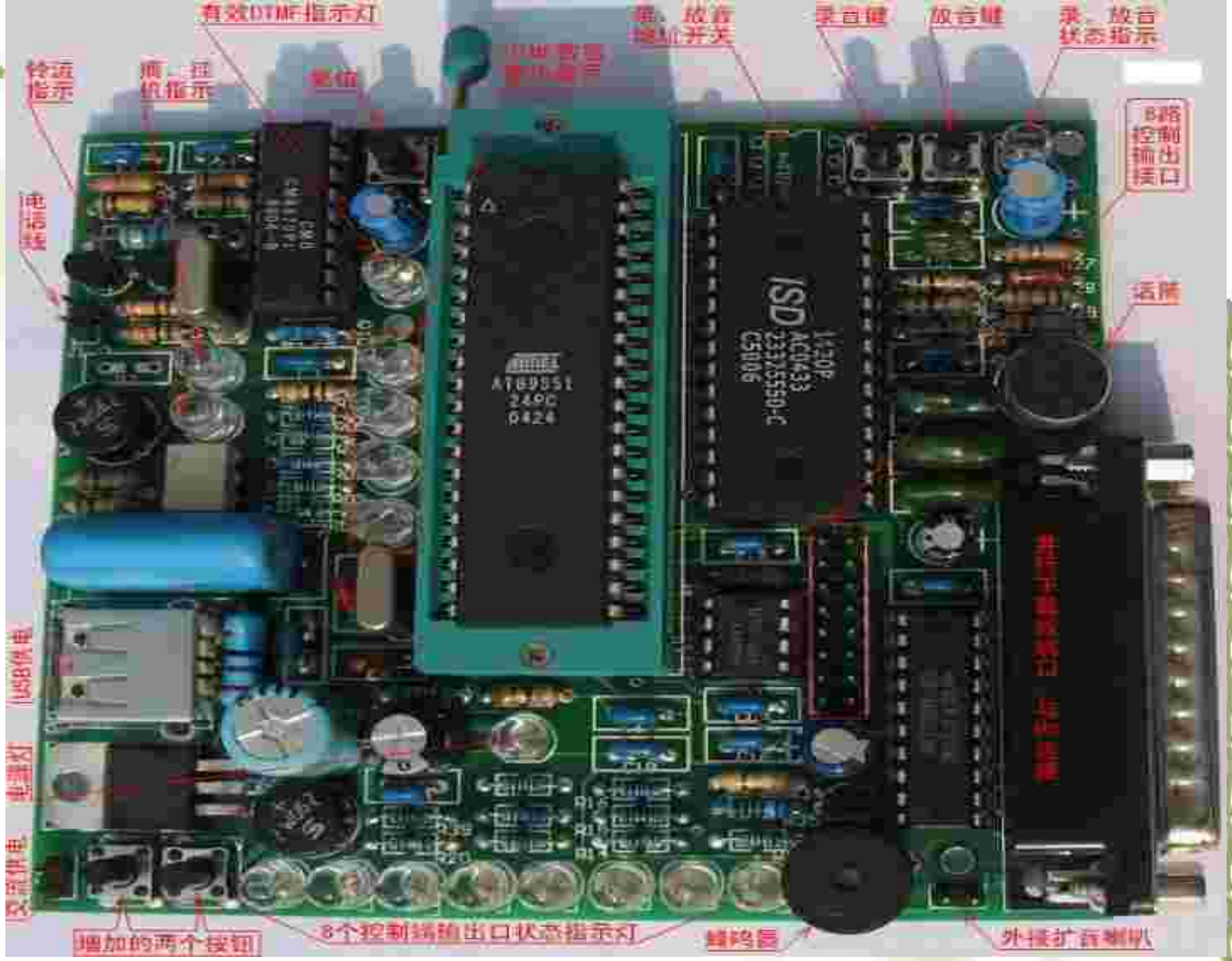
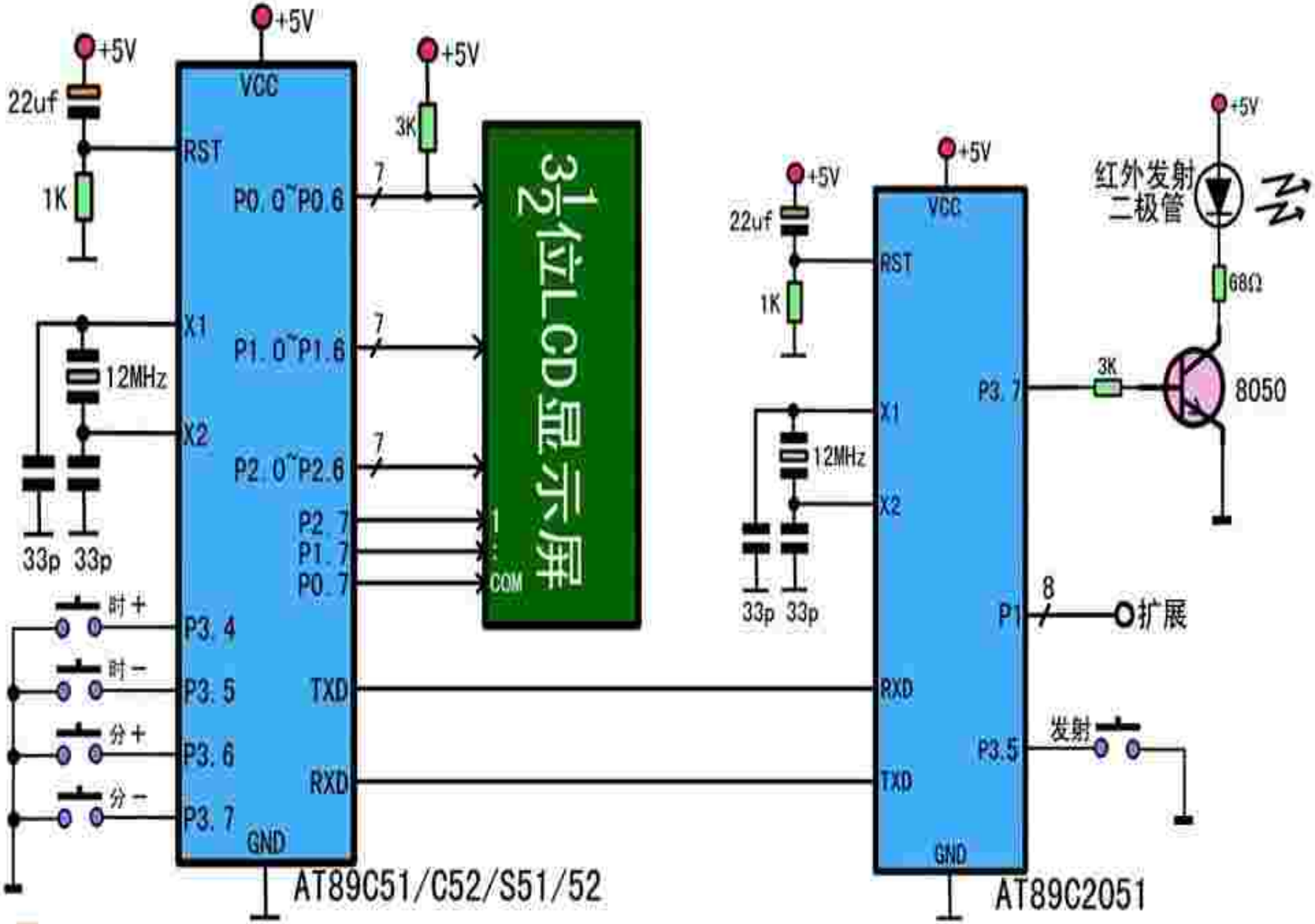
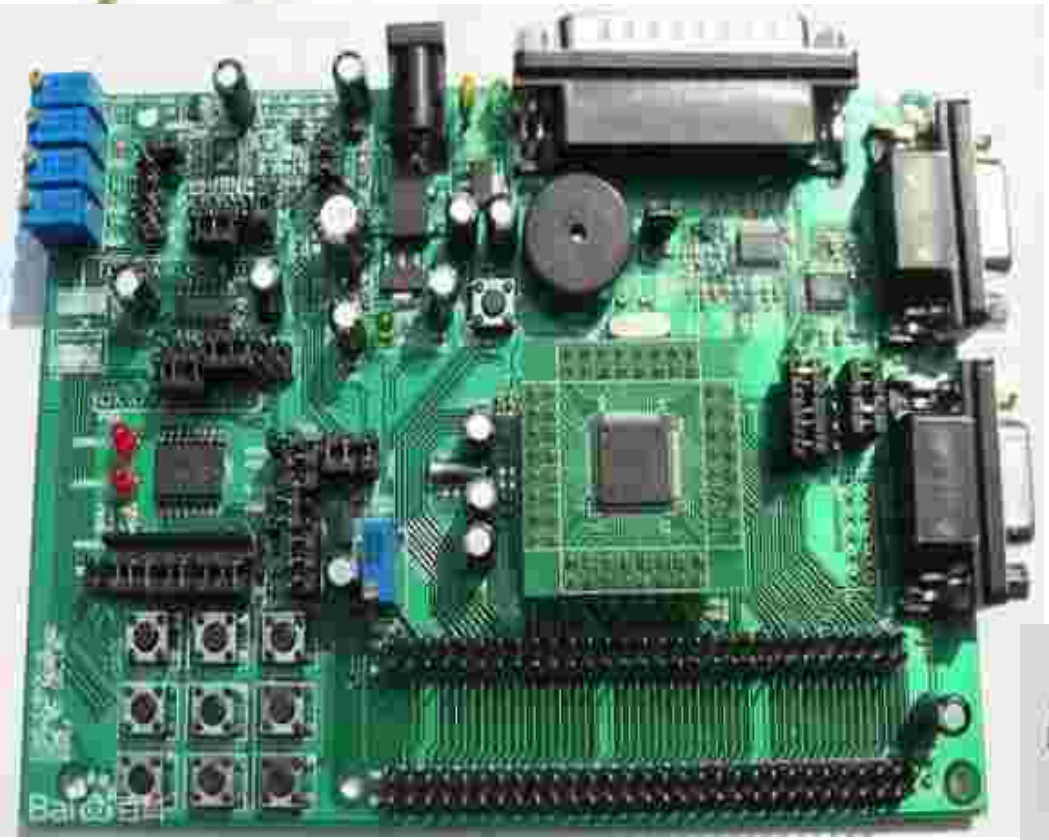

有效DTMF指示灯
录、放音
录音键
放音键
录、放音
状态指示
铃流
指示
摘、挂
机指示
8路
控制
输出
接口
电话线
话筒
(USB)供电
电源灯
交流供电
增加的两个按钮
8个控制输出口状态指示灯
蜂鸣器
外接扩音喇叭


+5V
22uf
1K
VCC
RST
P0.0~P0.6
X1
P1.0~P1.6
12MHz
X2
P2.0~P2.6
33p 33p
P2.7
P1.7
P0.7
时+
时-
分+
分-
P3.4
P3.5
P3.6
P3.7
TXD
RXD
GND
AT89C51/C52/S51/52
3K
7
1
32位LCD显示屏
COM
+5V
22uf
1K
VCC
RST
X1
12MHz
X2
33p 33p
P3.7
P1
8
扩展
RXD
TXD
P3.5
发射
GND
AT89C2051
红外发射
二极管
68Ω
3K
8050



MSP430开发板

MSP430系列单片机是美国德州仪器（TI）1996年开始推向市场的一种16位超低功耗、具有精简指令集（RISC）的混合信号处理器（Mixed Signal Processor）。

第1章 微型计算机原理基础知识

1.1 概述

§1.1.1 计算机的发展历史

1、计算机科学之父、英国著著名数学家阿兰·麦席森·图灵，于1939年提出著名的“图灵机”设想。

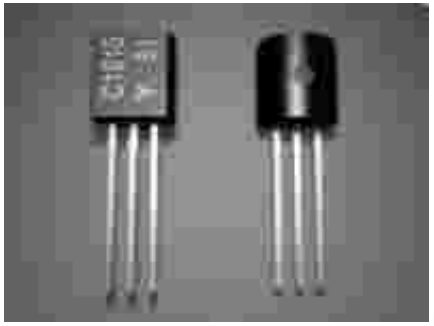
2、计算机之父冯·诺依曼，于1946年提出“存储程序控制”的现代计算机体系，即著名的冯.诺依曼计算机体系。

计算机发展，在短短的几十年里，经历了电子管计算机、晶体管计算机、集成电路计算机和大规模集成电路计算机。

电子管计算机→

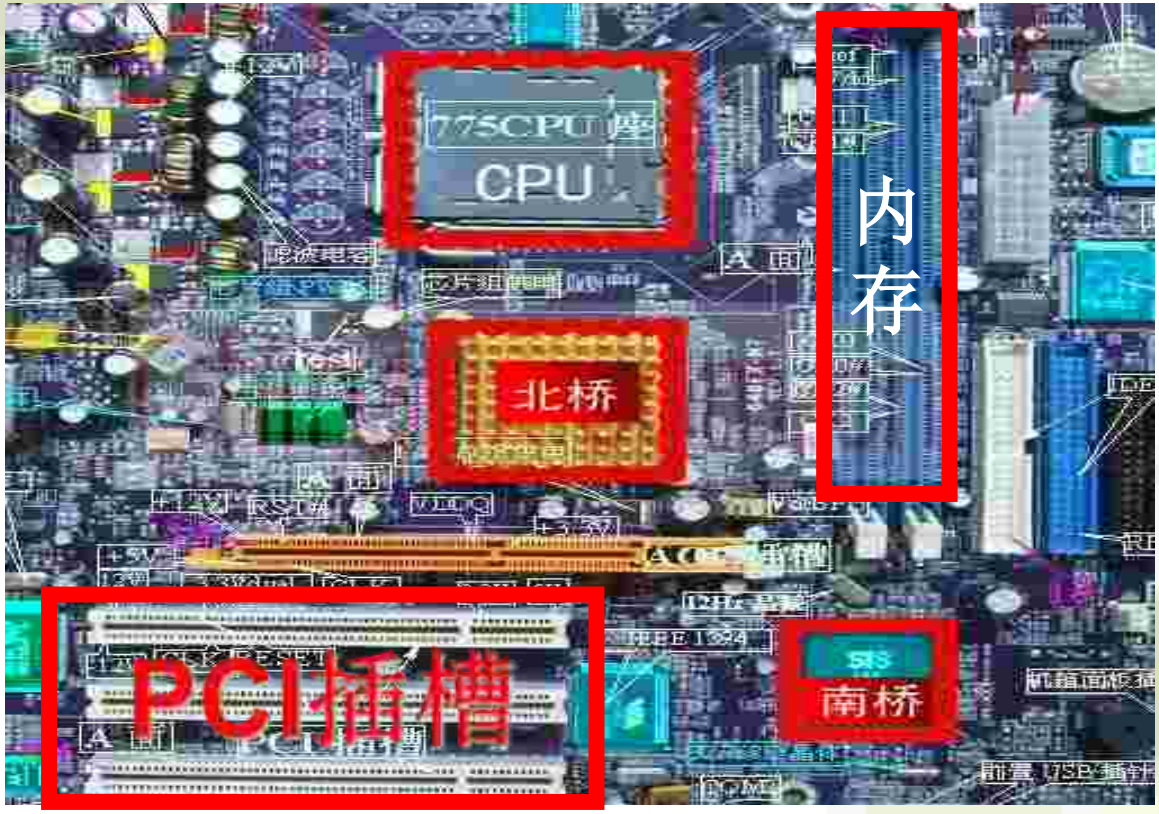
1945年第一代电子计算机（ENIA
个电子管，重30吨，占地150平方
次加法运算。

晶体管计算机→



集成电路计算机→

大（超大）规模集成电路计算机→



§1.1.3 单片微型计算机的发展历史

第一阶段(1976—1978)

以Intel公司首先推出的MCS-48系列单片微型计算机为代表。

第二阶段(1978—1982)

典型的MCS-51单片机系列，8位单片机。包括8位数据总线、16位地址总线、控制总线及具有多机通信功能的串行通信接口。

第三阶段(1982—1990)

在8位单片机的基础上，推出了16位单片机，也是单片机向微控制器发展的阶段。

第四阶段(1990之后)

随着单片机在各个领域全面、深入地发展和应用，出现了高速、大寻址范围、强运算能力的8位／16位／32位通用型单片机。

1.2 微型计算机中数制和数的表示

§1.2.1 数制

数值所使用数码的个数称为基；每一位所具有的值称为权。

- **十进制**的基为“10”，即它使用的数码为0，1，2，3，4，5，6，7，8，9共有10个。十进制各位的权是以10为底的幂，用“D”表示。

5	2	3	7	9	1
10^5	10^4	10^3	10^2	10^1	10^0
十万	万	千	百	十	个

- **二进制**的基为“2”，即其使用的数码为0、1共两个，用“B”表示。

二进制	1	1	0	1	1	1
	2^5	2^4	2^3	2^2	2^1	2^0
十进制	32	16	8	4	2	1

- **十六进制的基为“16”，即其数码共有16个：0，1，2，3，4，5，6，7，9，A，B，C，D，E，F。权为以16为底的幂，后缀“H”。**

§1.2.2 数制的转换方法

1、十进制数转换成二进制数

整数部分与小数部分分别转换

例1-1：57D=111001B=71Q=39H

2	57	
2	28…1	低
2	14…0	
2	7…0	
2	3…1	
2	1…1	高
	0…1	

8	57
8	7…1
	0…7

16	57
16	3…9
	0…3

除基数取余数

例：十进制数转换成二进制数（小数部分）

0.625D=0.101B=0.5Q=0.AH

0.625*2=1.25…1　高　0.625*8=5…5　0.625*16=10…A

0.25*2=　0.5…0

低

0.5*2=　　1…1

乘基数取整数

57.625D=111001.101B=39.AH

十进制小数要转换成二进制小数，则要采取“乘2取整法”（乘以基数正序取整）。

2、二进制数转换成十进制数⟶ 按权展开求和

例 1-3 求二进制数 101011 B 的十进制数。

	1	0	1	0	1	1
权：	2^5	2^4	2^3	2^2	2^1	2^0
乘积：	32	0	8	0	2	1

结果： 43 D

例 1-4 求二进制数 0.101 B 的十进制数。

	0	1	0	1
权：	2^0	2^{-1}	2^{-2}	2^{-3}
乘积：	0	0.5	0	0.125

结果： 0.625 D

3、十六进制数与二进制数之间的转换

二进制数→十六进制数 （四位一组）

十六进制数→二进制数 （一位对应换成二进制数四位）

分组时，如位数不够，整数部分在最左边补零，小数部分在最右边补零

例：把1101111100011.10010100B转换为十六进制数

0001	1011	1110	0011.	1001	0100
1	B	E	3	9	4

§1.2.3 计算机中数的表示

1、机器数和真值

机器数— 一个数在计算机中的表示形式。

真值 — 即为该机器数的实际算术值。

例：01001100B为机器数。 真值为+76

2、机器数的二进制表示

机器数可分为无符号数和有符号数。无符号数只能表示0和正数，有符号数能表示正数、负数和0。

1）无符号数的表示法

一个二进制数N可以表示为如下形式：

$$N_B = \sum_{i=-m}^{n-1} B_i \times 2^i$$

其中，n为二进制数N整数部分的位数；m为二进制数N小数部分的位数；B_i为二进制数字符号0或1。

例如：1101.1 B=$1\times2^3+1\times2^2+1\times2^0+1\times2^{-1}$。

2）带符号数的表示法

例如，当n=8时，

$[+1]_{原}$=00000001 B，　$[+127]_{原}$=01111111 B

$[-1]_{原}$=10000001 B，　$[-127]_{原}$=11111111 B

3、整型数的三种编码

1）原码

数x的原码记作$[x]$原，如机器字长为n，则原码定义如下：

$$[x]_{原}=\begin{cases} x & ,\quad 0\le x\le 2^{n-1}-1 \\ 2^{n-1}+|x|, & -(2^{n-1}-1)\le x\le 0 \end{cases}$$

2）反码

数x的反码记作$[x]$反，如机器字长为n，反码定义如下：

$$[x]_{反}=\begin{cases} x & ,\quad 0\le x\le 2^{n-1}-1 \\ (2^{n}-1)-|x|, & -(2^{n-1}-1)\le x\le 0 \end{cases}$$

3）补码

数x的补码记作$[x]$补，如机器字长为n，补码定义如下：

$$[x]_{补}=\begin{cases} x & ,\quad 0\le x\le 2^{n-1}-1 \\ 2^{n}-|x|, & -2^{n-1}\le x\le 0 \end{cases}$$

4、补码与真值之间的转换

求补运算是将一个二进制数按位求反加1的运算。

例：求下列数的补码。

① 设x=+127D，求$[x]_{补}$。

x=01111111B。$[x]_{补}=[+127]_{补}$=01111111B。

② 设x=-127D，求$[x]_{补}$。

对$[x]_{补}$进行求补运算便可得到$[-x]_{补}$。因此，

$[x]_{补}=[-127]_{补}=\overline{[+127]_{补}}+1=\overline{01111111\ B}+1=10000001\ B$。

例：求以下补码的真值。

① 设$[x]_{补}$=01111110 B，求x的真值。

该补码的最高位为“0”，该补码对应的真值是正数。

$x=[x]_{补}$=01111110 B =+126D。

② 设$[x]_{补}$=10000010 B，求x。

该补码的最高位为“1”，真值是负数，其绝对值为：

$|x|=\overline{[x]_{补}}+1=\overline{10000010}$ B+1=01111110B

=+126 D

则x= -126D。

1.3 计算机补码运算及运算电路

§ 1.3.1 补码运算

微机中的机器数一律用补码的形式表示。

1）两个数加法运算时，应将参加运算的数化成补码形式，相加后的结果仍为补码 。

2）对于有符号数来说，符号位无须单独处理，符号位可直接参加运算，不影响最终结果。

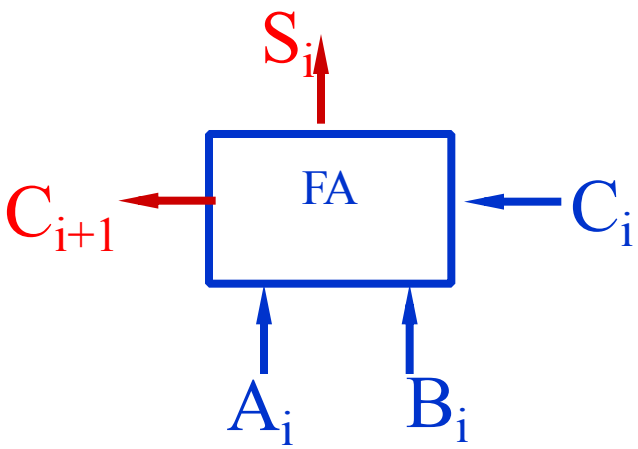
$$[X]_{补}+[Y]_{补}=[X+Y]_{补}$$

$$[X]_{补}-[Y]_{补}=[X-Y]_{补}$$

$$[X-Y]_{补}=[X]_{补}+[-Y]_{补}$$

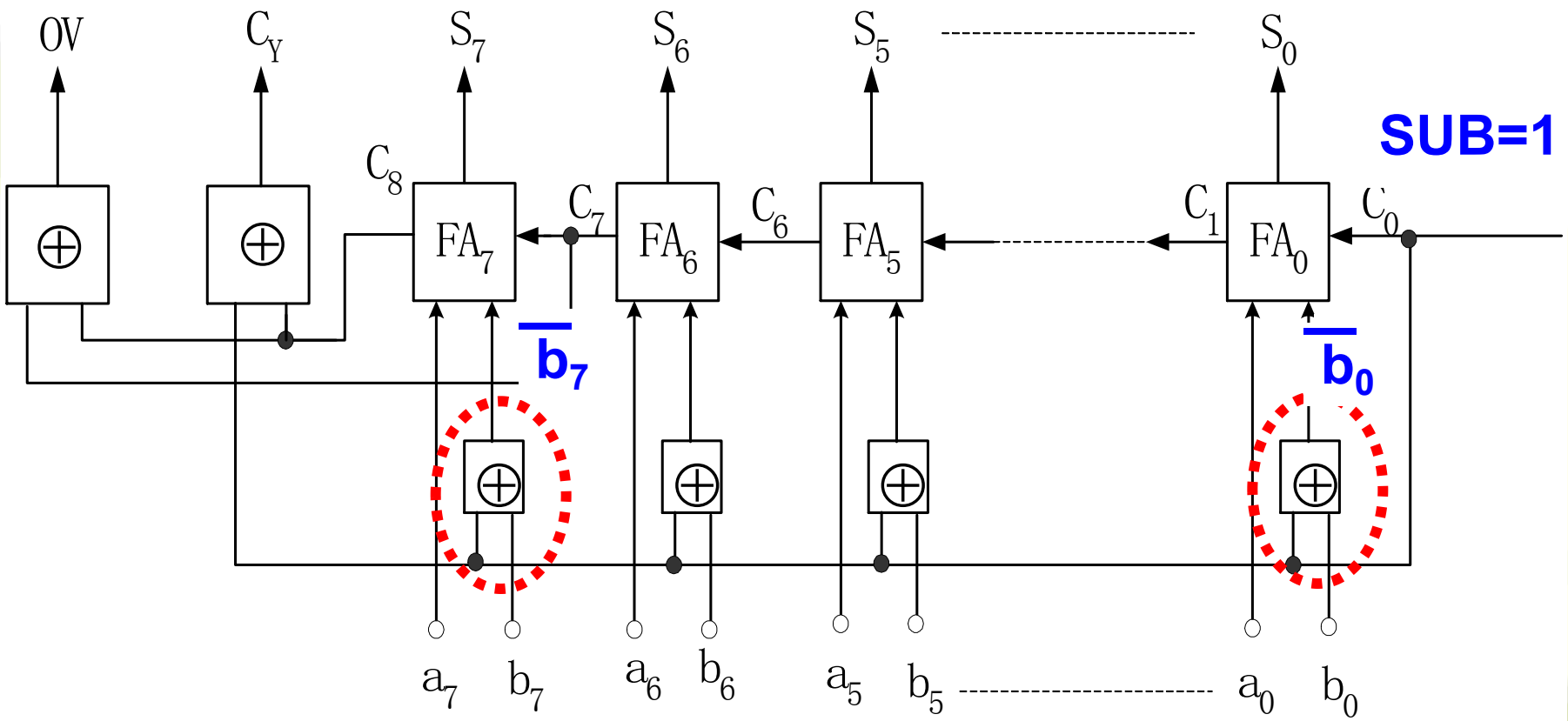
§1.3.2 计算机中的运算电路

微机中的运算都是在运算器中运行的，运算器是由加法器构成的，只会做二进制加法运算，一位二进制全加器电路如图所示：



全加器真值表

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$C_1C_2C_3C_4C_5C_6C_7C_8$ 为加法运算中低位向高位的进位

SUB为加减控制信号

$a_0a_1a_2a_3a_4a_5a_6a_7$ 为被加数/被减数　$b_0b_1b_2b_3b_4b_5b_6b_7$ 为加数/减数

$S_0S_1S_2S_3S_4S_5S_6S_7$ 为和或差

当执行加法运算指令时，SUB=0，图中各异或逻辑门均为同相逻辑门，对加数没有任何影响，则$S_i=a_i+b_i+c_i$。

当执行减法运算指令时，SUB=1，图中各异或逻辑门均为反相逻辑门，则$S_i= a_i-b_i+c_i = a_i+\overline{b_i}+c_i$，并且FA连同SUB的状态一起做加法运算。

结论： *减法运算是一个数减去另一个数，等于加上另一个数的反码并加1，这个运算过程称为补运算。*

减法：

$$[X]_{补} - [Y]_{补} = [X]_{补} + [-Y]_{补} = [X-Y]_{补}$$

图中Cy是进位（或借位）的状态标志位，做加法运算时若最高位（FA_7）有进位，则Cy=1，否则Cy=0；做减法运算时若FA_7位无进位，说明不够减，有借位现象，则Cy=1，否则，Cy=0

图中OV是溢出标志位，运算结果有溢出时，OV=1，否则OV=0，溢出的原因是运算结果超出了微机中数的表示范围。

$Cy=C_8 \oplus SUB$　　$OV=C_7 \oplus C_8$

§1.3.3 补码运算过程举例

加法运算（SUB=0）

例：35+42=　+77

$$
\begin{array}{rl}
 & [35]_{补} = 0\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\
+ & [42]_{补} = 0\ 0\ 1\ 0\ 1\ 0\ 1\ 0 \\
\hline
 & \boxed{0}\ 0\ 1\ 0\ 0\ 1\ 1\ 0\ 1 \\
 & C_8\quad C_7
\end{array}
$$

Cy=0，OV=0，结果为4DH，仍为补码形式。 若要显示运算结果，应对运算结果求真值。

真值是将补码形式的运算结果用“+”“–”号的形式来表示运算结果的实际数值。一般用十六进制或十进制表示。

例：85+78= ???

$$
\begin{array}{rl}
 & [85]_{补} = 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \\
+ & [78]_{补} = 0\ 1\ 0\ 0\ 1\ 1\ 1\ 0 \\
\hline
 & \boxed{0}\ 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\
 & C_8 \quad C_7
\end{array}
$$

Cy=0，OV=1，说明运算结果出现溢出现象，运算结果超出了微机中对数的表示范围。

例：76+(-23)= +53

$$
\begin{array}{rl}
 & [76]_{补} = 0\ 1\ 0\ 0\ 1\ 1\ 0\ 0 \\
+ & [-23]_{补} = 1\ 1\ 1\ 0\ 1\ 0\ 0\ 1 \\
\hline
 & \boxed{1}\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 1
\end{array}
$$

C_8 C_7

Cy=1，OV=0

减法运算（SUB=1）

例：已知X= 85，Y= 78，求 X - Y= +7

$[85]_{补}$= 0101 0101 B

[redacted]

补运算

\+ 1011 0010 B

$\boxed{1}$ 0000 0111 B

C_8 C_7=1

$CF=C_8 \oplus SUB = 1 \oplus 1 = 0$,

$OF=C_8 \oplus C_7 = 1 \oplus 1 = 0$,

减法运算（SUB=1）

例：已知X= + 76，Y= - 23，求 X-Y= + 99

$[76]_{补} = 01001100B$

$-$ [redacted]

$+ \quad 00010111B$

$01100011B$

补运算

进位$C_8 = 0$， $C_7=0$

$CF=C_8 \oplus SUB = 0 \oplus 1 = 1$，

$OF=C_8 \oplus C_7 = 0 \oplus 0 = 0$，

补码加法运算总结：

① 加数与被加数都采用补码形式，运算结果也是补码。

$$[X+Y]_{补}=[X]_{补}+[Y]_{补}$$

② 加数与被加数都看成带符号数，最高位为0，表示正数，最高位为1，表示负数

③ 当两个正数相加时，C_8肯定为0，若C_7=1，符号位变为1（负），得到一个负数，则发生溢出，OV=1（错误）；若C_7=0，符号位不变，则没有溢出，OV=0（正确）

问题：对于一些数据、字符、汉字等信息在微机中如何表示？

1.4 计算机中的常见编码

§1.4.1 ASCII码

ASCⅡ码是美国标准信息交换码，有两种版本。7位二进制数表示一个字符，早期用于电信行业，后被移植入微机，可表示128个字符。

96个图形字符
- 10个十进制数字
- 52 个英文字符
- 34 个其他字符

ASCⅡ码表中:

A～Z 的ASCⅡ编码是 41H～5AH

a ～z 的ASCⅡ编码是 61H～7AH

0 ～9 的ASCⅡ编码是 30H～39H

行	列	0	1	2	3	4	5	6	7
	MSB 位 654 / LSB 位 3210	000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC_1	!	1	A	Q	a	q
2	0010	STX	DC_2	"	2	B	R	b	r
3	0011	ETX	DC_3	#	3	C	S	c	s
4	0100	EOT	DC_4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	—	=	M	]	m	}
E	1110	SO	RS	.	>	N	↑	n	~
F	1111	SI	HS	/	?	O	←	o	DEL

§ 1.4.2 BCD码

在微机中用二进制数表示十进制数的方法很多，常用的有8421BCD码，即用四位二进制数表示一位十进制数，因组成它的4位二进制数码的权为8、4、2、1而得名。

8421BCD编码表

十进制数	8421 码	十进制数	8421 码
0	0000B	8	1000B
1	0001B	9	1001B
2	0010B	10	00010000B
3	0011B	11	00010001B
4	0100B	12	00010010B
5	0101B	13	00010011B
6	0110B	14	00010100B
7	0111B	15	00010101B

1. 压缩BCD码

是一个字节中存放两个十进制数字位。

如：将十进制数8962用压缩BCD码表示

1000　1001　0110　0010

在主存中存放形式：

10001001
01100010

2. 非压缩BCD码

是指每个字节中只存放一个十进制数字位。

如：将十进制数8962用非压缩BCD码表示

0000 1000　0000 1001　0000 0110　0000 0010

在主存中存放形式：

00001000
00001001
00000110
00000010

3. BCD码加法运算

BCD加法就是按“逢十进位”的原则进行相加，其和也是一个BCD数。但微机只能进行二进制加法，在相邻BCD码字之间只能“逢十六进位”。导致出现非BCD码。因此需要对结果进行调整，做到“逢十进位”。

例： 34+62= 96

$$
\begin{array}{r}
[34]_{BCD}=00110100B \\
+\ [62]_{BCD}=01100010B \\
\hline
10010110B
\end{array}
$$

← 结果仍为BCD码

例： 48+69= 117

$[48]_{BCD}$ =0100 1000B

\+ $[69]_{BCD}$ =0110 1001B

1011 0001B ← 低位向高位产生进位，高位出现非BCD码

\+ 0110 0110B ← 均应作加6调整

进位 → 10001 0111B ← 结果仍为BCD码

相加结果为无符号数，最高位进位有效

4. BCD码减法运算

减法运算的调整原则：在运算过程中某位出现借位或非BCD码，相应位作减6调整，减去多借的6。也是有专门的指令来完成的。

例： 58-25= 33

$$
\begin{array}{rr}
 & [58]_{BCD}=01011000B \\
- & [25]_{BCD}=00100101B \\
\hline
 & 00110011B
\end{array}
$$

← 结果仍为BCD码

例： 35-28= 7

$[35]_{BCD}$=00110101B ←低四位向高四位借位

– $[28]_{BCD}$=00101000B

00001101B ←运算结果的低4位出现非BCD码

– 00000110B ← 个位做减6调整

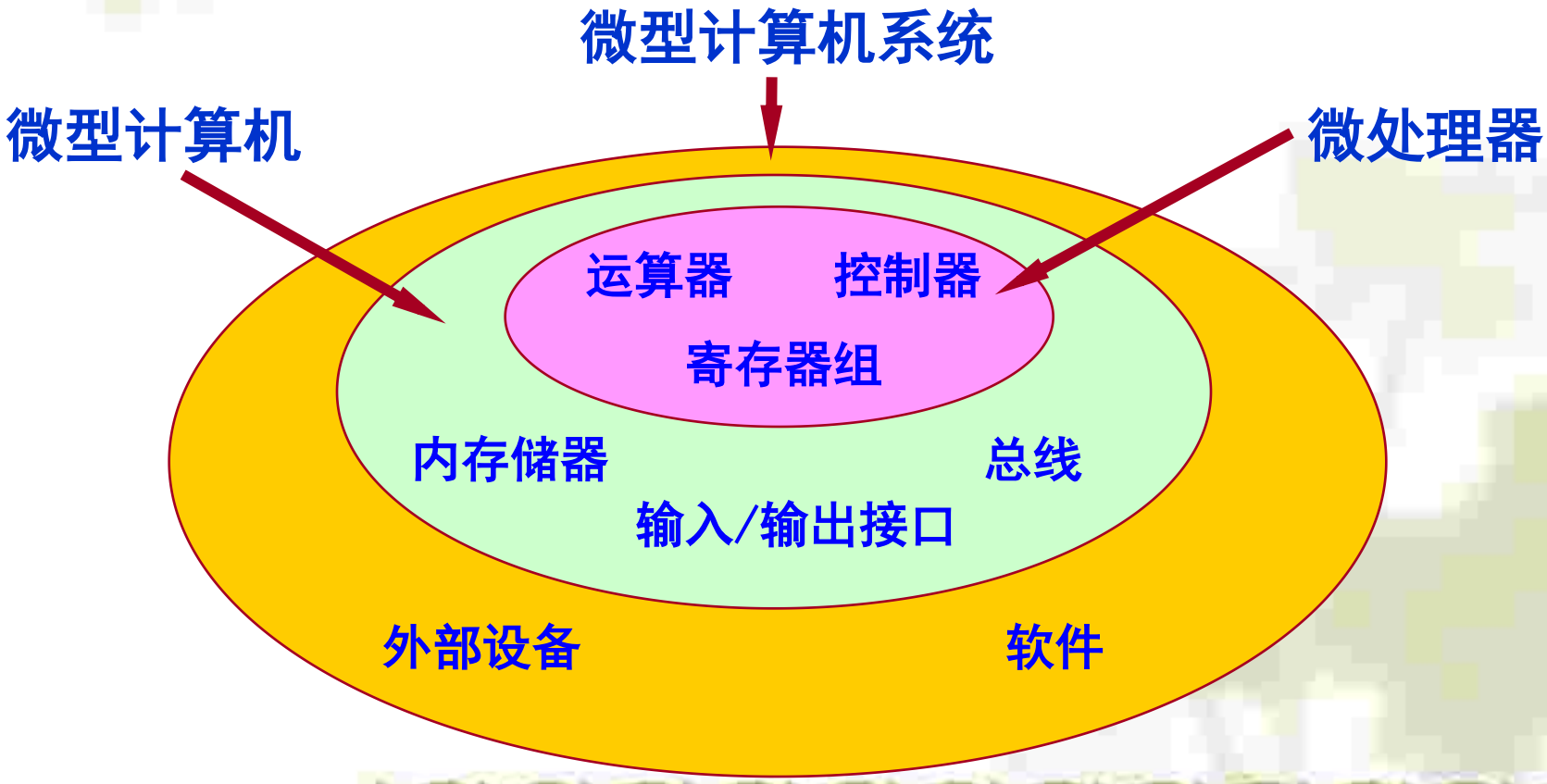
00000111B ← 结果仍为BCD码

1.5 微型计算机系统

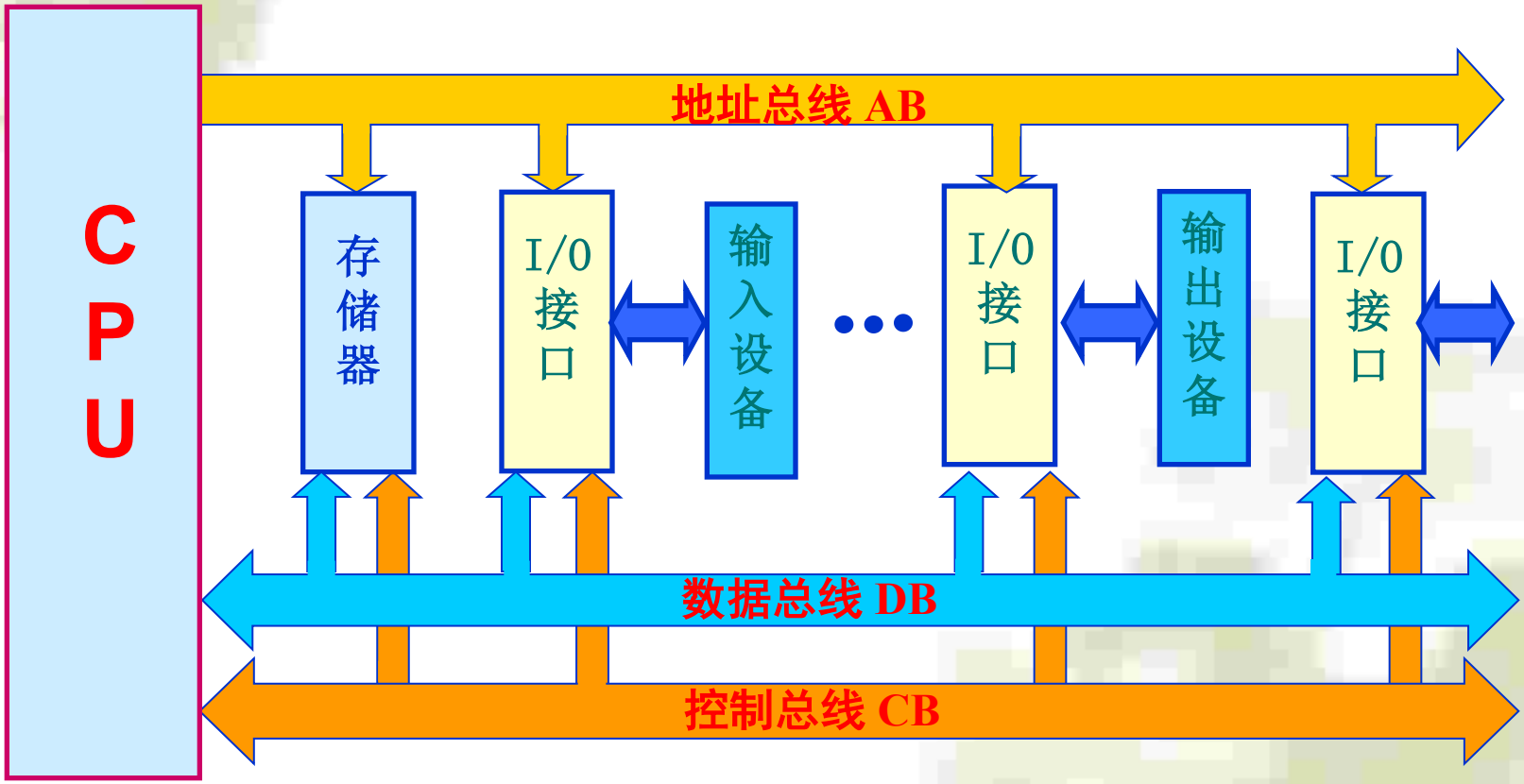
- 微型计算机系统
  - 硬件系统
    - 主 机
      - 微处理器（CPU）
        - 运算器（算术逻辑部件ALU）
        - 控制器（控制部件CU）
        - 寄存器阵列（RA）
      - 内存储器
      - 输入/输出接口电路
      - 系统总线
    - 外围设备
      - 输入设备（键盘、鼠标等）
      - 输出设备（显示器、打印机等）
      - 外存储器（磁带、磁盘和光盘等）
  - 软件系统
    - 系统软件
      - 操作系统
      - 系统应用程序
    - 应用软件

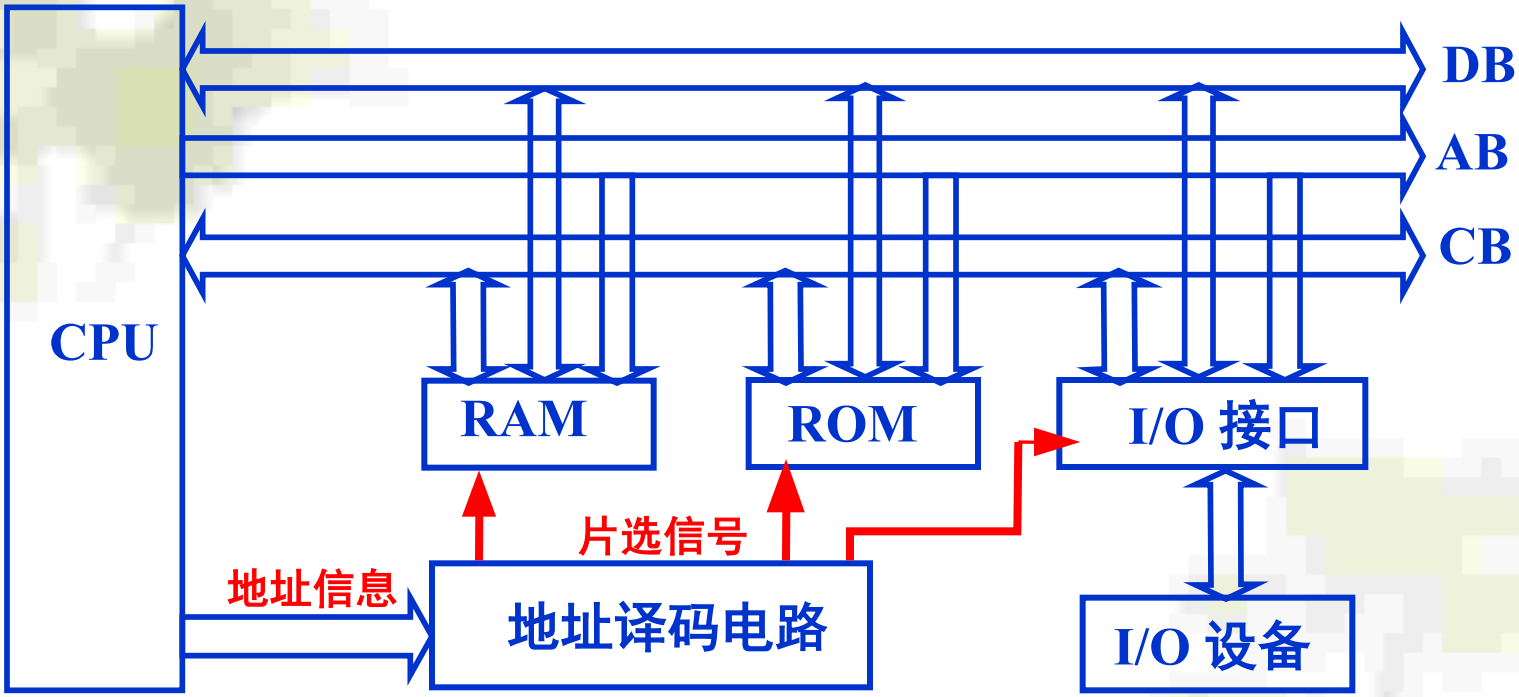
§ 1.5.1 微型计算机主机组成

微型计算机系统的组成可分为微处理器、微型计算机、微型计算机系统三个层次。



微型计算机的概念结构





主机结构突出特点是模块化结构。

总线结构，系统中各部件 “挂”在总线上。

1. 微处理器

MPU(微处理器) → 嵌入系统中与其他电路及芯片组合称为CPU

微机的核心部件，主要由运算器、控制器和寄存器组构成。

- 运算器（ALU）—主要负责算术运算和逻辑运算。
- 控制器（CU）—指令寄存器、指令译码器和控制电路组成。并按指令的要求对微机各部件发出相应的控制信息。
- 工作寄存器组—暂存寻址和计算过程的信息.

  地址寄存器——地址寄存器用于操作数的寻址。

  数据寄存器——数据寄存器用来暂存操作数和中间运算结果。

2．内存储器（主存）

用于“记忆”信息的存储元件，均采用集成度高，容量大，体积小，功耗低的半导体存储器芯片。

1）存储单元：字节（Byte) 基本单位，8位二进制数称为一个字节。

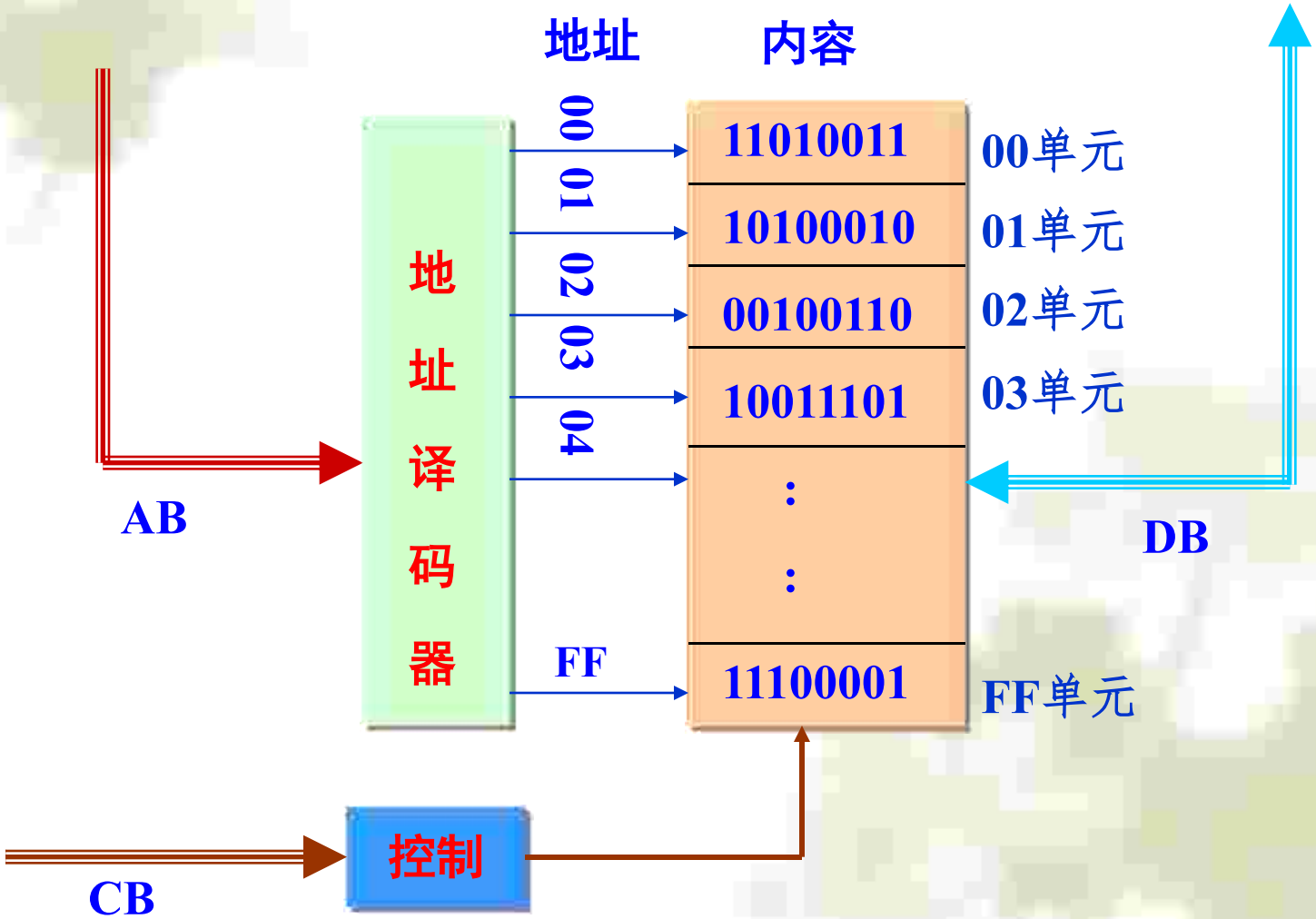
例如： 0100 1101 B

一个字节数据空间称为一个存储单元，其中每一位称为1Bit。

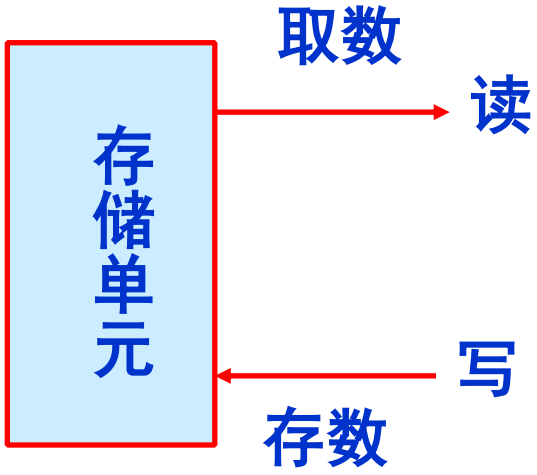
2）存储地址

微机的存储器是由许多存储单元集合而成，每一个存储单元唯一的对应一个单元地址，也称为存储器的物理地址，微机中的地址一般都用十六进制表示，如30H。

存储器：用于存放程序代码及有关数据



CPU执行访问内存的指令时，按指定的单元地址对相应存储单元进行“读、写”操作。



存储器容量（s）=存储单元数（p）×数据位数（i）

1024B=1KB　　1024KB=1MB

1024MB=1GB　　1024GB=1TB

3. 总线 → 传送信息的公共导线

根据传送信息的内容与作用不同，总线分为：

① 数据总线DB：双向传输数据信息

其宽度（根数）与MPU提供的数据线的引脚数有关，表示微机的字长。

② 控制总线CB：传送各种控制信号和状态信号，单向传送。

③ 地址总线AB：CPU执行指令时，用于单向传送地址信息。

地址信息
- 指令代码在存储器中的地址信息
- 操作数在存储器中的地址信息

AB的宽度决定了微机系统的最大寻址能力。

最大寻址空间=2^N，其中N为AB的宽度

例：

8086/8088CPU N=20

最大寻址空间 2^{20}=1MB

51单片机 N=16

最大寻址空间 = 2^{16} = 65536 B = 64 KB

4. 地址译码电路

凡是“挂”在总线上部件都被系统分配一个地址域，CPU访问时，由指令提供被访问部件的地址信息，该地址信息经地址译码电路译码后唯一的产生一个选通信号（称片选信号），将被选中的部件“门”打开，使得数据得以传输。

5. 接口

接口是主机与外设连接的必然通路，是必经的“桥梁”。每个接口可包含若干个端口，每个端口对应一个端口地址，可由指令按地址访问端口。

§1.5.2 微型计算机的主要性能指标

1. 字长

8086/8088CPU内部寄存器为16位，所以字长为16位，称为16位机；80386、80486、80586（pentium）字长均为32位，故均称为32位机。

字长越长的微机，其运算速度越快，数的表示范围越宽，数据的运算精度越高，机器的整体功能越强。

2. 存储器容量

存储二进制信息的基本单位是位（b）。一般把8个二进制位组成的基本单元为字节（B）。

微机中通常以字节为单位表示存储容量。

1024B =1KB

1024KB =1MB（兆字节）

1024MB =1GB（吉字节）

1024GB =1TB（太字节）

3. 运算速度

微机的运算速度一般用每秒钟所能执行的指令条数来表。

4.系统总线

系统总线的性能主要表现为它所支持的数据传送位数和总线工作时钟的频率。数据传送位数越多，总线工作时钟频率越高，则系统总线的信息吞吐率就越高，微机系统的性能就越强。

5. 外设扩展能力

外设扩展能力是指微机系统配接各外部设备的可能性、灵活性和适应性。

6. 软件配置情况